



Moduli e Package Python

Advanced Computer Programming

Prof. Raffaele Della Corte



Sources

- <http://docs.python.org/library/>
 - Informazioni sulla libreria standard di Python
- <http://docs.python.org/tutorial/modules.html>
 - Tutorial sui moduli di Python



Cosa sono i moduli Python?

- Le definizioni di funzioni e variabili non vengono salvate all'uscita dell'interprete
- I moduli consentono di salvare le definizioni per accedervi in un secondo momento
- I moduli consentono inoltre di eseguire le istruzioni come script eseguibili
- I moduli sono file con estensione .py



Importare un modulo

- I moduli vengono importati utilizzando il comando incorporato `import`, senza l'estensione `.py`
`>>> import example`
- Per rendere più facile l'accesso, si possono importare singole definizioni all'interno di un modulo
`>>> from function import func1, func2`
- Quando i moduli sono importati, tutte le dichiarazioni e le definizioni saranno eseguite



Accedere ai componenti di un modulo

- Per utilizzare le funzioni definite nel modulo, digitare il nome del modulo seguito dal punto

```
>>> example.func(3)
```

- Se la funzione nel modulo è stata importata individualmente con *from*, il nome del modulo e il punto possono essere omessi

```
>>> func(3)
```



Moduli come script

- Gli script Python con estensione .py possono essere eseguiti nella riga di comando con:

```
python example.py <arguments>
```

- I moduli importati e quelli eseguiti possono essere distinti accedendo alla variabile globale **__name__**:
 - il valore di `__name__` è "**__main__**" solo quando i moduli sono eseguiti



Moduli come script

- Le istruzioni di controllo possono essere utilizzate come segue:

```
if __name__ == "__main__":
```

- Tutto il codice contenuto nell'istruzione if non verrà eseguito se il modulo viene importato



Search path per i moduli

- L'interprete cerca tutti i moduli importati in determinati luoghi designati, nell'ordine seguente, finché il modulo non viene trovato:
 - Directory corrente
 - L'elenco delle directory nella variabile d'ambiente `PYTHONPATH`
 - Percorso predefinito dipendente dall'installazione



Search path per i moduli

- La variabile d'ambiente `PYTHONPATH` è un elenco di directory
- La directory corrente, la variabile d'ambiente `PYTHONPATH`, e il *default path* (dipendente dall'installazione) sono tutte informazioni memorizzate nella variabile `sys.path`



Moduli standard

- Esiste una libreria standard di moduli Python
- Questi moduli contengono operazioni integrate che non fanno parte del nucleo di Python
- Alcuni moduli vengono importati solo se si utilizza un determinato sistema operativo
- Un modulo di libreria importante è **sys**



Il modulo *sys*

- Due variabili contenute in *sys*, *ps1* e *ps2*, contengono i valori delle stringhe nei prompt primario e secondario
- Queste stringhe possono essere modificate a piacimento

```
>>> import sys
```

```
>>> sys.ps1 = 'plurt> '
```

```
plurt> print 'plurt'
```

```
plurt
```

```
plurt>
```



La funzione *dir()*

- La funzione `dir()` restituisce un elenco di nomi (variabili e funzioni) definiti dal modulo ad essa passato.

```
>>> dir(sqrt)
```

```
['num', 'result']
```

- Quando `dir` viene eseguito senza argomenti, restituisce un elenco di tutti i nomi attualmente definiti, ad eccezione di quelli incorporati



Packages

- I ***package*** sono raccolte organizzate di moduli, moduli all'interno di moduli
- I moduli sono memorizzati in directory, con ogni directory che contiene un file `__init__.py`
 - Il file `__init__.py` istruisce l'interprete Python di trattare la directory come se fosse un *package*
 - Il file `__init__.py` può essere semplicemente un file vuoto, ma può anche eseguire codice di inizializzazione per il *package* o impostare la variabile `__all__` (trattata più avanti)
- Ogni sottodirectory della directory principale del pacchetto è considerata un sottopacchetto



Packages

- Il *package* o uno qualsiasi dei suoi sotto-package può quindi essere importato

- E.g.:

import **currency.conversion.euro**

- È ora possibile accedere a qualsiasi definizione o sottopacchetto di **euro**, ma solo utilizzando la notazione punto:
 - **euro.convertfromdollar(10)**



Importazione da un package

- A volte si vuole importare tutto ciò che si trova all'interno di un package o di un sottopackage. In questo caso si può usare questo comando:

from example.subexample import *

- Questo carica tutti i nomi dei moduli contenuti nella variabile `__all__`, a sua volta contenuta nel file `__init__.py` della cartella del pacchetto



Importazione da un package

- Se la variabile `__all__` non è definita, per impostazione predefinita, l'importazione tramite `*` da un package
 - Non importerà tutti i sottomoduli dal package
 - Assicurerà di importare tutto il package, possibilmente eseguendo il codice di inizializzazione contenuto in `__init__.py` e importerà tutti i nomi definiti del package
 - Eventuali sottomoduli non saranno importati, a meno che non siano esplicitamente importati da `__init__.py`



Riferimenti Intra-package

- Quando i package sono strutturati in sottopackage, si possono usare le **importazioni assolute** per fare riferimento ai sottomoduli dei package “Fratelli”
 - E.g., se il modulo `sound.filters.vocoder` deve usare il modulo `echo` del pacchetto `sound.effects`, può usare `from sound.effects import echo`.
- È anche possibile scrivere **importazioni relative**, con la forma di dichiarazione `from module import name`
 - Queste importazioni utilizzano dei punti iniziali per indicare il pacchetto corrente e quello padre coinvolti nell'importazione relativa
 - E.g.:
 - `from . import echo`
 - `from .. import formats`
 - `from ..filters import equalizer`



Packages in directory multiple

- La posizione in cui un package cerca il suo `__init__.py` può essere cambiata, modificando la variabile `__path__`, che memorizza il valore della cartella in cui si trova `__init__.py`